LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – Relasi-Antar-Kelas



Aditya Dwi Aryanto

103132400027

SD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Diagram kelas merupakan salah satu jenis diagram yang digunakan untuk memodelkan kelas-kelas di dalam sistem beserta hubungan antar kelas tersebut. Diagram ini memuat elemen penting berupa nama kelas, atribut, dan method, serta menggunakan simbol *access modifier* seperti + untuk *public*, – untuk *private*, dan # untuk *protected*. Dalam pemrograman berorientasi objek, kelas-kelas yang terbentuk dapat saling terhubung melalui beberapa bentuk relasi utama, yaitu asosiasi, agregasi, komposisi, dan generalisasi.

Relasi asosiasi terjadi ketika dua kelas terpisah berkomunikasi dengan cara memanggil method milik kelas lain tanpa menjadikannya sebagai atribut permanen. Sementara itu, agregasi menggambarkan hubungan antara bagian (*part*) dan keseluruhan (*whole*), di mana kelas bagian tetap dapat diciptakan dan berdiri sendiri terpisah dari kelas keseluruhan. Di sisi lain, komposisi merupakan bentuk agregasi yang lebih ketat, sebab keberadaan kelas bagian bergantung sepenuhnya pada kelas keseluruhan sehingga jika kelas keseluruhan dimusnahkan, maka kelas bagian ikut dimusnahkan. Selain itu, terdapat pula relasi generalisasi yang merepresentasikan konsep pewarisan (*inheritance*), di mana suatu kelas menjadi turunan dari kelas lainnya.

B. Guided

1. Agregasi

Jurusan.java

```java
package guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan;
    private String namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    // constructor
    public Jurusan(String kodeJurusan, String namaJurusan) {
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    // behavior
    public void addMahasiswa(Mahasiswa m) {
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa() {
        System.out.println("Kode Jurusan " + this.kodeJurusan);
```

```
        System.out.println("Nama Jurusan " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa : ");

        for (int i = 0; i < jumlahMahasiswa; i++) {
            System.out.println((i + 1) + ". " +
daftarMahasiswa[i].getNIM() + " " + daftarMahasiswa[i].getNama());
        }
    }

    // getter
    public String getKodeJurusan() {
        return kodeJurusan;
    }

    public String getNamaJurusan() {
        return namaJurusan;
    }

    // setter
    public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan) {
        this.namaJurusan = namaJurusan;
    }
}
```

Mahasiswa.java

```
package guided.agregasi;

public class Mahasiswa {
    private String NIM;
    private String nama;

    public Mahasiswa(String NIM, String nama) {
        this.NIM = NIM;
        this.nama = nama;
    }

    // getter
```

```
    public String getNIM() {
        return NIM;
    }

    public String getNama() {
        return nama;
    }

    // setter
    public void setNIM(String NIM) {
        this.NIM = NIM;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }
}
```

2. Main

   Main.java

```
package Main;
import guided.agregasi.Jurusan;
import guided.agregasi.Mahasiswa;
import guided.komposisi.Komputer;

public class main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("103132400027", "Aditya");
        Mahasiswa m2 = new Mahasiswa("103132400028", "belang");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }
```

```
    public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
" Main.main "
Kode Jurusan SD
Nama Jurusan Sains Data
Daftar Mahasiswa :
1. 103132400027 Aditya
2. 103132400028 belang
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
```

3. Asosiasi

Mahasiswa.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java
to edit this template
 */
package guided.Asosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
```

```
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }

    public int getNilai() {
        return this.nilai;
    }
}
```

Dosen.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java
to edit this template
 */
package guided.Asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }
```

```
    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        // ini asosiasi, method milik class Mahasiswa dipanggil di
class Dosen
        // tapi objek Mahasiswa tidak menjadi atribut dari class
Dosen
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        // ini asosiasi, method milik class Mahasiswa dipanggil di
class Dosen
        // tapi objek Mahasiswa tidak menjadi atribut dari class
Dosen
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :

```
psids-05-reg-01\mo
Nim :130118383
Nama :Budi
Nilai :90
```

4. Komposisi
   CPU.java

```java
package guided.komposisi;

public class CPU {
    private String merk;
    private int kecepatan;

    public CPU(String m, int k) {
        this.merk = m;
        this.kecepatan = k;
    }

    public void DisplaySpecCPU() {
        System.out.println(this.merk + ", " + this.kecepatan);
    }
}
```

Komputer.java

```java
package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

Monitor.java

```java
package guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

Mouse.java

```java
package guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Prosesor.java

```java
package guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
```

```
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

C. Unguided

1. Soal Unguided 1

   Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang tidak dapat berdiri sendiri. Siklus hidup objek internal terikat penuh pada objek utamanya. Pada sistem cafe: - Setiap item pesanan (ItemPesanan) pasti memiliki rincian harga (RincianHarga) yang mencatat harga dasar dan pajak (10%). - Objek RincianHarga harus di-instansiasi langsung di dalam constructor ItemPesanan. Objek RincianHarga tidak pernah dibuat di main. Jika ItemPesanan

   Jawaban :

   RincianHarga.java

```
package unguided.Komposisi;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.1;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
```

```
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

ItemPesanan.java

```
package unguided.Komposisi;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

Penjelasan :
Pada tugas pertama, relasi komposisi diterapkan antara kelas ItemPesanan sebagai objek utama dan RincianHarga sebagai bagian internal yang siklus hidupnya terikat penuh pada objek utamanya. Kelas RincianHarga menyimpan atribut hargaDasar dan menghitung pajak sebesar 10% dari harga dasar, serta menyediakan method getHargaTotal() untuk menjumlahkan keduanya. Prinsip komposisi diwujudkan dengan menginstansiasi objek RincianHarga menggunakan kata kunci new secara langsung di dalam constructor ItemPesanan (bukan di main), sehingga objek RincianHarga tidak dapat berdiri sendiri dan otomatis ikut terhapus apabila objek ItemPesanan dihapus.

2. Soal Unguided 2

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang dapat berdiri sendiri. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah. Pada sistem cafe: - Sebuah Pesanan menampung beberapa objek ItemPesanan menggunakan array standar. - Objek ItemPesanan dibuat terpisah terlebih dahulu (di main), lalu dimasukkan satu per satu ke dalam Pesanan melalui method tambahItem(). Jika Pesanan dibatalkan atau dihapus, objek ItemPesanan tersebut secara konsep tetap ada.

Jawaban :

Pesanan.java

```java
package unguided.Agregasi;

import unguided.Komposisi.ItemPesanan;
import unguided.Komposisi.RincianHarga;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
            System.out.println("Kapasitas pesanan penuh.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);
        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga rincian = item.getRincian();
```

```
            System.out.println((i + 1) + ". " + item.getNamaItem() +
" - Rp " + rincian.getHargaTotal()
                    + " (Harga: " + rincian.getHargaDasar() + ",
Pajak: " + rincian.getPajak() + ")");
        }
        System.out.println();
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

Penjelasan :
Tugas kedua mengimplementasikan relasi agregasi melalui kelas Pesanan yang bertindak sebagai objek wadah untuk menampung kumpulan objek ItemPesanan ke dalam array bertipe tetap (ItemPesanan[] daftarItem). Berbeda dengan komposisi, objek ItemPesanan dibuat secara terpisah di luar kelas Pesanan, lalu dimasukkan satu per satu ke dalam array menggunakan method tambahItem() disertai pencatatan indeks pada atribut jumlahItem. Karena bersifat agregasi, keberadaan objek ItemPesanan secara konsep tetap ada meskipun objek Pesanan dibatalkan, dan isi pesanan tersebut dapat dicetak ke layar melalui method tampilkanPesanan()

3. Soal Unguided 3
   Asosiasi adalah relasi interaksi antara dua objek independen yang saling menggunakan
   tetapi tidak saling memiliki. Objek tidak menyimpan objek lain sebagai field permanen,
   melainkan berinteraksi melalui parameter method.
   Pada sistem cafe:
   - Seorang Kasir memproses pembayaran dari sebuah Pesanan.
   - Class Kasir tidak menyimpan objek Pesanan sebagai field, melainkan menerima objek Pesanan sebagai parameter pada method prosesPembayaran().

Jawaban :

Kasir.java

```java
package unguided.Asosiasi;

import unguided.Agregasi.Pesanan;
import unguided.Komposisi.ItemPesanan;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public String getNamaKasir() {
        return namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftar = pesanan.getDaftarItem();
        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            totalSubtotal += daftar[i].getRincian().getHargaDasar();
            totalPajak += daftar[i].getRincian().getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja " +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " +
pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("------------------------------");
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
```

```
}
```

Penjelasan :
Pada tugas ketiga, relasi asosiasi diterapkan pada kelas Kasir untuk menggambarkan interaksi antara dua objek independen yang saling menggunakan tanpa saling memiliki. Kelas Kasir hanya memiliki atribut namaKasir dan dilarang menyimpan objek Pesanan sebagai *field* permanen di dalam kelasnya. Hubungan keduanya hanya terjadi saat method prosesPembayaran(Pesanan pesanan) dipanggil, di mana Kasir menerima objek Pesanan melalui parameter method untuk membaca daftar item, menghitung akumulasi subtotal beserta pajak, dan mencetak struk pembayaran

4. Soal Unguided 4
   Buatlah class Main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.
   Jawaban :

   Main.java

```java
package unguided.Main;

import unguided.Komposisi.ItemPesanan;
import unguided.Agregasi.Pesanan;
import unguided.Asosiasi.Kasir;

public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);

        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
------------------------------
TOTAL BAYAR    : Rp 71500.0
==============================
```

Penjelasan :

Tugas keempat merangkai seluruh kelas di dalam method main untuk menguji ketiga relasi objek sesuai skenario operasional kafe. Alur program dimulai dengan membuat satu objek Kasir ("Budi") dan tiga objek ItemPesanan ("Iced Americano", "Beef Croissant", dan "Matcha Latte") yang secara otomatis memicu relasi komposisi pada rincian harganya. Selanjutnya, dibuat objek Pesanan untuk Meja 4 yang menampung ketiga item tadi lewat method tambahItem() sebagai bentuk agregasi, diikuti pemanggilan tampilkanPesanan() untuk menampilkan daftar menu dan prosesPembayaran(pesanan) oleh objek Kasir sebagai bentuk asosiasi.

D. Kesimpulan

Diagram kelas merupakan elemen krusial dalam pemrograman berorientasi objek yang berfungsi untuk memodelkan struktur kelas beserta hubungan antar kelas di dalam suatu sistem. Tingkat keterikatan antar kelas dalam sistem tersebut bervariasi, mulai dari asosiasi yang sebatas interaksi pemanggilan method, agregasi yang membentuk hubungan bagian dan keseluruhan namun tetap saling independen, hingga komposisi yang memiliki keterikatan sangat kuat di mana keberadaan kelas bagian bergantung sepenuhnya pada kelas keseluruhan. Selain itu, terdapat relasi generalisasi yang menerapkan konsep pewarisan antar kelas. Dengan memahami dan menerapkan jenis relasi yang tepat, perancangan arsitektur perangkat lunak dapat dilakukan secara lebih terstruktur, fleksibel, dan efektif sesuai dengan kebutuhan sistem.